

ABSTRACT

The main aim of the project to develop a system automatic speed control of vehicle and accident avoidance using ultrasonic sensor. Whenever any obstacle is detected in running vehicle depends on distance automatically control the speed of vehicle. The ultrasonic sensor system continuously sends signals and monitors any car or other obstacles are in front of car. The distance up to which ultrasonic sensor can work may be up to 4meter. When any obstacle or vehicle detected by ultrasonic sensor system it will send signal to the embedded board. After receiving this signal embedded board sends a signal to the motor to reduce the car speed automatically which can control car speed immediately. Vehicle is controlled automatically without any manual operation when the vehicle is at 4meter distance away from the front vehicle. Also give alarm to alert to the driver. Many accidents at High-ways are taking place due to the close running of vehicles, all of sudden, if the in front vehicle driver reduces the speed or applied breaks, then it is quite difficult to the following vehicle driver to control his vehicle, resulting accident. To avoid this kind of accident, the warning system, which contains alarm and display system can arrange at rear side of each and every vehicle

Key Words: Speed control, Sensors, Security, Electronic stability control, Velocity control, Speed limitation, Driver Assistance System

INDEX

CHAPTER	CONTENT	PAGE NO
CHAPTER-1	INTRODUCTION	1-3
	1.1 OBJECTIVE OF THE PROJECT	1
CHAPTER-2	LITERATURE SURVEY	4-6
CHAPTER-3	3.1 BLOCK DAIGRAM	7-12
	3.2 DESCRIPTION OF THE BLOCK DIAGRAM	8
CHAPTER-4	AVR MICROCONTROLLER	13-26
	4.1 ARUDINO	13
	4.2 HARDWARE	15
CHAPTER-5	FUNCTIONAL MODULES	27-36
	5.1 POWER SUPPLY	27
	5.2 DESCRIPTION	33
CHAPTER-6	SOFTWARE MODULE DESCRIPTION	37-39
CHAPTER-7	PROJECT IMPLEMENTATION	40-42
	7.1 IMPLEMENTATION	40
	7.2 ADVANTAGES AND APPLICATIONS	40
	CONCLUSION	44
	RESULT	45
	REFERENCES	46-47
	APPENDIX	48-50

LIST OF FIGURES

FIG.NO	NAME OF THE FIGURE	PAGE.NO
Fig 3.1	Block diagram of project	7
Fig 4.1	Arduino UNO	14
Fig 4.2	Arduino UNO hardware	15
Fig 4.3	Arduino UNO Connections	21
Fig 4.4	Arduino UNO Pinout diagram	22
Fig 4.5	Atmega 168 Pin mapping	21
Fig 5.1	Components of Power supply	27
Fig 5.2	LED Display	29
Fig 5.3	DC Motors	31
Fig 5.4	DC Motors working	32
Fig 5.5	Pin diagram of L293D	35

LIST OF TABLES

TABLES	Description	Page. No.
Table 4.1	COMPONENTS AND THEIR RANGE	19
Table 5.1	FUNCTIONS OF PIN DIAGRAM	30
Table 5.2	PIN DESCRIPTION	34

ACRONYMS

VSC	Vehicle Speed Control
ADAS	Advanced Driver-Assistance Systems
AV	Autonomous Vehicle
ITS	Intelligent Transportation Systems
IVS	Intelligent Vehicle System
CAD	Computer Aided Design
CDMA	Code Division Multiple Access
CPU	Central Processing Unit
DC	Direct Current
DTMF	Dual Tone Multi Frequency
GMS	Globalization Management System
GPS	Global Positioning System
IC	Integrated Circuit
LCD	Liquid Crystal Display
MATLAB	Matrix Laboratory
PIR	Passive Infrared Sensor
PROM	Programmable Read Only Memory
OTP	One Time Password
SMS	Short Service Message
RAM	Random Access Memory
ROM	Read Only Memory
RPM	Rotation Per Minute
RS	Reset
TCON	Timer Control Register
TMOD	Timer Module
Euro NCAP	European New Car Assessment Programme
ISO	International Organization for Standardization

ECU	Electronic Control Unit
GPS	Global Positioning System
IMU	Inertial Measurement Unit
LIDAR	Light Detection and Ranging
RADAR	Radio Detection and Ranging
CAN	Controller Area Network
OBD	On-Board Diagnostics
NHTSA	National Highway Traffic Safety Administration

CHAPTER 1

INTRODUCTION

1.1 Objectives of project

DC motors have many applications in many fields of industrial, commercial and other activities, such as robotics, automobiles, servomechanisms etc. The electric drive systems used in many industrial applications require higher performance, reliability, variable speed due to their ease of controllability. The speed control of a DC motor is crucial in applications where precision and protection are essential. The purpose of a motor speed controller is to take a signal representing the required speed and to drive a motor at that speed. Microcontrollers can provide easy control of a DC motor. A microcontroller-based speed control system consists of an electronic component and a microcontroller. There are many applications of DC motor drives that use power electronics to control the voltage and consequently the speed or position of the motor. For large motors it is highly economical to use power electronics, in order to minimize the power loss and the size of the motor. DC choppers are easily implemented. In this paper, an Arduino UNO system (microcontroller) was used for the control signal. The DC motor fed by a DC chopper that was driven by the Arduino UNO microcontroller. The chopper is driven by a high frequency PWM signal. Controlling the PWM duty cycle is equivalent to controlling the motor terminal voltage, which in turn directly adjusts the motor speed.

This work is a practical one and highly feasible from an economic and accuracy point of view. The desired objective is to achieve a system running on a desired constant speed at any load condition. That means the motor will run on a fixed speed instead of varying with the amount of load.

In the field of electronics and control systems, it is often required to control the speed of a DC motor with precision. One of the popular ways to achieve this is by using an Arduino Uno board and PWM (Pulse Width Modulation) signal to control the motor speed. This report describes the development of a system that utilizes an Arduino Uno board, an L293D IC, and LM35 temperature sensor to control the speed of a DC motor and monitor the temperature of the system. The DC motor speed in general is directly proportional to the supply voltage, so if reduce the voltage from 9 volts to 4.5 volts then our speed become half of what it originally had. But in practice, for changing the speed of a dc motor we cannot go on changing the supply voltage all the time. Pulse width modulation speed control works by driving the motor with a series of “ON-OFF” pulses and varying the duty cycle, the fraction of time that the output voltage is “ON” compared to when it is “OFF”, of the pulses while keeping the frequency constant.

The speed controller PWM for a DC motor works by varying the average voltage supplied to the motor. PWM signal is essentially a high frequency square wave (typically greater than 1KHz). The PWM controller accepts the control signal and adjusts the duty cycle of the PWM signal according to the requirements.

The **Vehicle Speed Control Project** is a technology-driven solution aimed at improving road safety, enhancing fuel efficiency, and reducing accidents caused by speeding. This project integrates various sensors, electronic control units (ECUs), and communication technologies to monitor and regulate the speed of a vehicle automatically, depending on traffic conditions, road types, and other relevant factors.

The primary goal of the vehicle speed control system is to provide an intelligent mechanism that maintains a vehicle's speed within predefined limits. This system helps prevent over-speeding, which is a leading cause of accidents, and can be adapted for both private cars and commercial vehicles to ensure smoother traffic flow and greater compliance with traffic regulations.

The DC motor speed in general is directly proportional to the supply voltage, so if reduce the voltage from 9 volts to 4.5 volts then our speed become half of what it originally had. But in practice, for changing the speed of a dc motor we cannot go on changing the supply voltage all the time.). The Duty Cycle of this square wave is varied in order to vary the power supplied to the load. The input signals we given to PWM controller might be an analog or digital signal according to the design of the PWM controller.

The Vehicle Speed Control Project represents an essential advancement in intelligent transportation systems (ITS). It not only improves road safety but also enhances driving comfort, environmental sustainability, and fuel efficiency. As vehicle technology evolves, such systems are likely to become more advanced, incorporating artificial intelligence and machine learning to make real-time driving decisions that protect drivers and pedestrians alike.

Background The "Automated Vehicle Speed Control System" project provide smart way to decrease road accident by controlling the speed, this system control vehicle acceleration when vehicle reach the limit. All kind of road speed limit data will be send to the system which is installed in vehicle. Easy to find out criminal who want to escape. We are find out smart solution for all kind of vehicle such as car, bus, motorcycle and more.

Vehicle Speed Control Mechanization Framework is intense, adaptable and extremely simple to utilize. It is planned and created to reduce traffic jam, road accident and crime to make our life comfortable. In the event that we check out us we will find that innovation is making its place all over. From morning to night, we are utilizing such a significant number of innovations,

in short it is a piece of our life now and it is extremely difficult to live without it too. Vehicle Speed Control Mechanization is winding up increasingly prevalent step by step because of its various favorable circumstances and that is the reason we picked this plan to make it more reliable and we try to much more accessible in our nation as well. We endeavored to control this speed with IR, IR is available, cheaper rate and very small also it is the fastest way to send data. For example, ©Daffodil International University 2 noise free hospital and school area, automated bus gate lock system and etc. Our last objective with this item is to bring our nation one stage ahead in innovation and nation fulfillment.

CHAPTER 2

LITERATURE SURVEY:

The PID controller is generated by an Arduino program and used in the speed control of DC motor. The objective of this paper is to control the speed of the DC motor using Arduino. Arduino board plays the role of low-cost data acquisition board. LM393 is a sensor which measures the revolutions.[1] Android apps provide a seamless interface for control, monitoring and visualization. Bluetooth terminal application and Arduino compiler was used to operate and control the switching action of the D.C motor of 5V.

It was observed that we are able to control the D.C motor from a range of Bluetooth device within the range of ten meters.[2] In this paper, the speed control of Permanent Magnet DC (PMDC) motor is done using Lab VIEW interfaced with Arduino. The main advantage of using Lab VIEW with Arduino is the cost and simple in structure. The other speed control methods like FPGA method, fuzzy control, using 555 timer and PID controllers are having the drawback of complicated design involved, unreliable control, difficult in the online monitoring with high-speed motors. These draw backs motivated us to work on Lab VIEW with Arduino.[3] In this paper we describe a technical system for DC motor speed control. The speed of DC motor is controlled using Arduino programming platform and MATLAB's Simulink coder. This paper contains introduction to using an Arduino board and Simulink PI controller in closed loop system.

It will be described how to program Arduino with Simulink coder and in the end, we present the results of PI controller for DC motor speed will be given.[4] Arduino represents the receiver side interface 8 channel relay module with 8-bit RF receiver. It is isolated from transmitter side. Figure 8 represents the DC SSR interface with DC Geared motor. Arduino Uno microcontroller board is interfaced with the RF transmitter side. Console is connected with personal computer through USB cable. DC SSR is placed between H-Bridge and DC source. It is operated by PWM signals from Arduino Uno board.[5]

Several approaches have been researched in terms of controlling vehicle speed:

- **Cruise Control Systems:** Traditional cruise control allows drivers to set a desired speed that the vehicle will maintain automatically. Advanced variants, such as adaptive cruise control (ACC), can adjust the speed based on the distance from the preceding vehicle and road conditions.
- **Speed Limiters:** These systems ensure that vehicles do not exceed a set speed limit. Speed

limiters are often used in commercial vehicles and are mandatory in certain jurisdictions.

- These systems may operate through the engine control unit (ECU) to limit the power available to the engine.
- **Intelligent Speed Assistance (ISA):** ISA systems use a combination of GPS, cameras, and maps to detect speed limits on the road and adjust the vehicle's speed accordingly. These systems may provide visual or audible warnings, or even automatically reduce the speed of the vehicle when the speed limit is exceeded.
- **Autonomous Speed Control:** In autonomous vehicles, speed control is integrated with other vehicle management systems, using sensors, AI, and real-time data (e.g., traffic, road conditions) to regulate speed automatically in a way that ensures safety and efficiency.

Several key technologies are integrated into modern vehicle speed control systems:

- **GPS and Mapping Systems:** GPS is used for determining vehicle location and cross-referencing this with digital maps to adjust speed based on road conditions, speed limits, and traffic regulations.
- **Cameras and Sensors:** Cameras detect road signs, including speed limits and environmental conditions, while sensors (e.g., radar, lidar) monitor the distance and speed of surrounding vehicles to maintain safe distances.
- **Vehicle-to-Everything (V2X) Communication:** V2X technology enables vehicles to communicate with each other and infrastructure (e.g., traffic signals, road signs) to adjust speed and improve safety. It can also provide real-time traffic information for more dynamic speed regulation.
- **Machine Learning and AI:** AI-based systems learn from driving patterns, road conditions, and traffic behavior to predict optimal speeds, further enhancing the adaptability of speed control systems. Machine learning is also used in developing predictive models for optimal speed regulation in autonomous vehicles.

While vehicle speed control technologies have seen significant advancements, several challenges persist:

- **Driver Overriding:** Many speed control systems, especially those that provide warnings or semi-automatic control, can be overridden by the driver. This undermines the effectiveness of the system and its ability to prevent speeding.
- **Sensor and Mapping Accuracy:** GPS and sensor systems may occasionally provide inaccurate data due to poor signal quality or sensor malfunctions. This can lead to incorrect speed adjustments or misinterpretations of speed limits.

- **Driver Acceptance and Trust:** Fully autonomous speed control systems often face resistance from drivers who may not trust the system or prefer to maintain manual control over vehicle speed. Balancing automated control with user trust is a key challenge.
- **Infrastructure and Communication Issues:** V2X communication relies on the availability of infrastructure that supports it (e.g., equipped traffic signals and vehicles). The implementation of such infrastructure is still limited in many regions.

Several recent studies and innovations have made significant contributions to vehicle speed control systems:

- **Adaptive Cruise Control (ACC):** ACC systems, including enhanced versions using radar and lidar, adjust vehicle speed in real time based on traffic flow. Research has focused on improving the accuracy of these systems in complex traffic environments.
- **Collision Avoidance Systems:** Many speed control systems now integrate with collision avoidance technologies, automatically adjusting speed to prevent potential accidents by maintaining safe following distances.
- **Smart Traffic Signals and Vehicle Coordination:** Research has explored how smart traffic signals, coupled with vehicle-to-infrastructure (V2I) communication, can help vehicles adjust their speeds proactively in response to traffic flow, congestion, and accidents.
- **Eco-Speed Control:** Some modern systems optimize vehicle speed not only for safety but also for fuel efficiency. These systems adjust the vehicle's speed to minimize fuel consumption based on real-time driving conditions and road gradients.

Proposed system features:

This automated vehicle speed control system will be able to have the following features for the good traffic system. 1. Speed Control: These systems can be program for vehicle speed control. We can do this in two ways. Automatically: This means that speed can be automatically control by IR sensor when the vehicle detects IR value the speed can be change automatically. Manually: This means the traffic authority can be change the IR value manually, 2. Call alert and Security system: The main objective of this part of our project is to send alert message or call to the authority if the diverse want to violate the rules and try to speed up to three times.

CHAPTER 3

3.1 Block diagram:

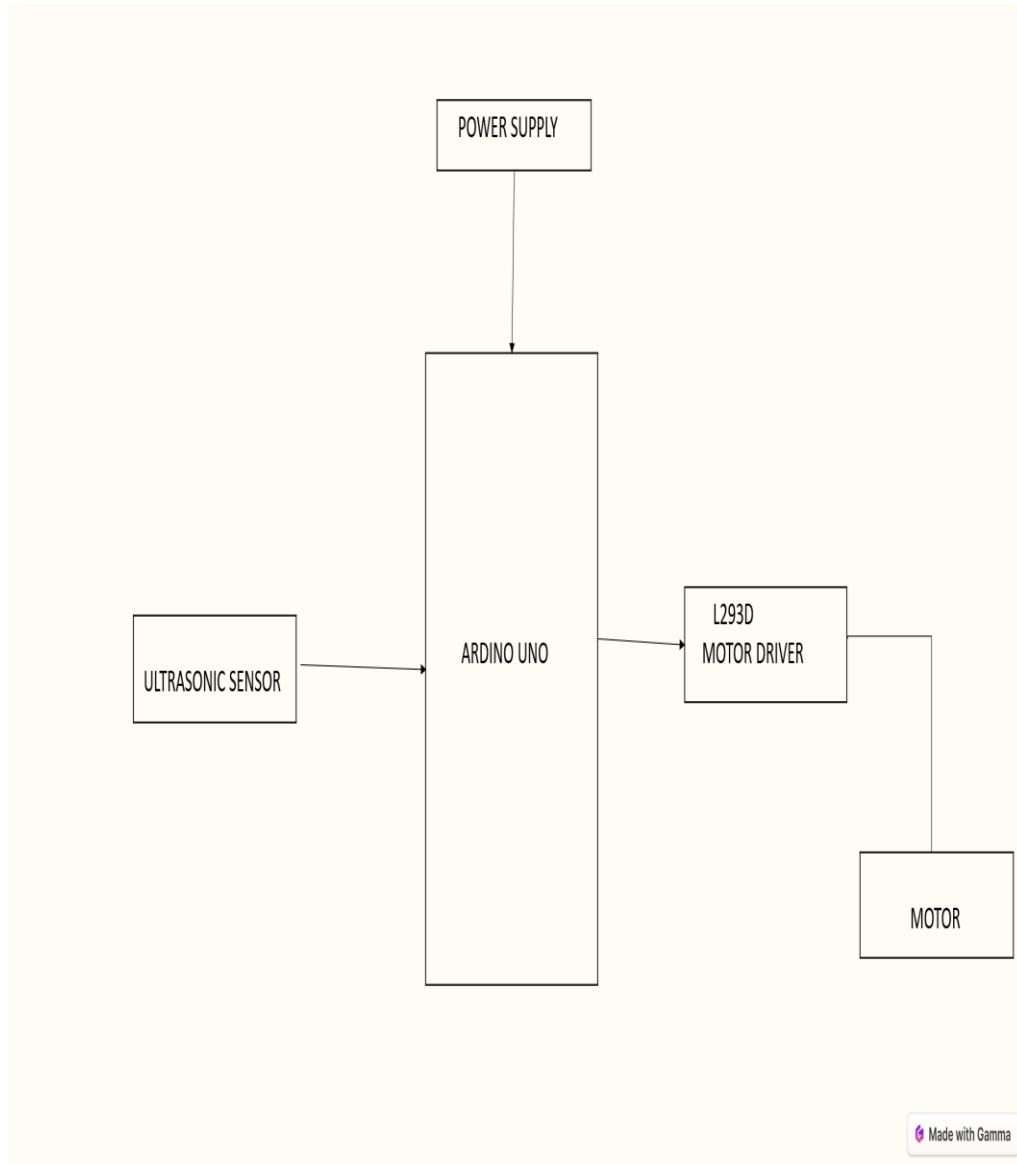


Figure 3.1: Block diagram of the project

Selection of Implementation Platform Selection of Implementation Platform There are several platforms over that a automated Vehicle Speed control system is enforced. Of the presently accessible platforms: Arduino, ATmega328p microcontroller, Android Device, IR sensor, GSM , microcontroller ATmega2560; Bluetooth Module and Microcontroller were found most appropriate due to their low price accessibility, reliableness and ease once used for a personal automated Vehicle Speed control system during which my project work

is on.

Selection of Hardware Components Each platform has a set of hardware components over which it is implemented. For ATmega328p microcontroller, For Bluetooth, there are many more Bluetooth module, but Bluetooth module HC-06 was chosen due to its low cost availability, it can be a category 2 slave Bluetooth module designed for clear wireless serial communication. Once it's paired to a master Bluetooth device like laptop computer, smart phones and pill, its operation becomes clear to the user. Finally, for Microcontroller, the popular ones are units those created by Microchip, ATMEL, Motorola and TX Instruments, of these chip manufactured Arduino microcontroller was found best suited due to its low cost. For IR sensor/Infrared will use several platform, the paired unit contains associated RF receiver and IR transmitter. Once associated infrared remote is used on the IR receiver, the device interprets the signal and broadcasts it over RF. The paired unit then receives that signal, decodes it and transmits associated IR signal. Associated IR transmitter contains associated crystal rectifier that emits infrared.

3.2 Description of the block diagram:

Vehicle Speed Control (VSC) is a critical technology designed to regulate and manage the speed of vehicles, ensuring safe and efficient transportation. With the increasing number of vehicles on the road, VSC has become essential for preventing accidents, reducing traffic congestion, and enhancing driver experience.

There are several types of VSC systems, including Adaptive Cruise Control (ACC), Electronic Speed Control (ESC), Intelligent Speed Adaptation (ISA), Automatic Speed Limiters (ASL), and Dynamic Speed Control (DSC). Each type has its unique features and applications, but all share the common goal of controlling vehicle speed.

A VSC system consists of several key components, including sensors, control units, actuators, and communication systems. Sensors detect speed, distance, and road conditions, while control units process sensor data and control the vehicle's speed. Actuators adjust speed through throttle, brakes, and transmission.

The primary functions of VSC systems include speed monitoring, automatic braking, collision avoidance, traffic congestion management, and fuel efficiency optimization. These functions work together to ensure safe and efficient transportation.

VSC systems offer numerous benefits, including improved road safety, reduced traffic congestion, enhanced fuel efficiency, lower emissions, and increased driver convenience. By regulating vehicle speed, VSC systems reduce the risk of accidents and

minimize environmental impact.

VSC systems are applied in various vehicles, including passenger vehicles, commercial vehicles, autonomous vehicles, public transportation systems, and smart city infrastructure. As transportation technology advances, VSC will play a vital role in shaping the future of transportation.

The technologies used in VSC systems include Artificial Intelligence (AI), Machine Learning (ML), Internet of Things (IoT), Computer Vision, and Sensor Fusion. These technologies enable VSC systems to detect and respond to changing road conditions.

Despite its benefits, VSC faces challenges such as cybersecurity concerns, sensor reliability and accuracy, integration with existing infrastructure, public acceptance and adoption, and regulatory frameworks. Addressing these challenges will ensure widespread adoption of VSC technology.

Future advancements in VSC will focus on integration with autonomous driving systems, advanced sensor technologies, AI-powered predictive maintenance, vehicle-to-everything (V2X) communication, and smart city infrastructure development. These advancements will further enhance road safety, reduce traffic congestion, and improve driver experience.

The name *Arduino* comes from a bar in Ivrea, Italy, where some of the founders of the project used to meet. The bar was named after Arduin of Ivrea, who was the margrave of the March of Ivrea and King of Italy from 1002 to 1014.

A VSC system consists of several key components, including sensors, control units, actuators, and communication systems. Sensors detect speed, distance, and road conditions, while control units process sensor data and control the vehicle's speed. Actuators adjust speed through throttle, brakes, and transmission.

In conclusion, Vehicle Speed Control systems are crucial for ensuring road safety, reducing traffic congestion, and enhancing driver experience. As transportation technology continues to evolve, VSC will remain a vital component of intelligent transportation systems, shaping the future of transportation.

DC motors are commonly used for controlling the speed of small vehicle components like fans, steering, and other auxiliary systems. In an electric vehicle or hybrid vehicle, DC motors are also used to directly control the vehicle's movement. The speed of these motors can be adjusted via a motor controller, making them essential for speed control applications.

The speed of a DC motor can be controlled by varying the voltage supplied to the motor. In speed control systems, feedback mechanisms (such as encoders or tachometers)

are used to monitor and adjust the speed in real time to ensure the vehicle is traveling at the desired speed

Ultrasonic sensors are used in various vehicle systems, especially for measuring distance, detecting obstacles, and controlling speed based on the proximity of surrounding objects. In a speed control system, they play a role in collision avoidance, adaptive cruise control, and maintaining safe distances between vehicles.

Function of Ultrasonic Sensors

- **Distance Measurement:** Ultrasonic sensors use sound waves to measure the distance between the sensor and nearby objects (such as other vehicles, barriers, or obstacles in the path). These sensors emit a high-frequency sound pulse and measure the time it takes for the pulse to reflect back after hitting an object. Using this data, the sensor calculates the distance to the object.
- **Speed Control Role:** The ultrasonic sensor can be used to dynamically adjust the vehicle's speed based on the distance to an obstacle or another vehicle. For example:
 - **Adaptive Cruise Control:** In systems like adaptive cruise control (ACC), ultrasonic sensors monitor the distance between the vehicle and the one in front. If the vehicle detects that it is getting too close to the vehicle ahead, the speed control system can reduce the throttle or apply brakes to maintain a safe distance.
 - **Collision Avoidance:** Ultrasonic sensors are widely used in collision avoidance systems. If the vehicle is approaching an obstacle at an unsafe speed, the sensor triggers a response (such as braking or reducing acceleration), helping to prevent an accident.
 - **Park Assist:** In lower-speed operations, ultrasonic sensors are used for parking assistance, where they detect objects in the vehicle's path and notify the driver of potential obstacles. This function may also be linked to speed control mechanisms for automatic parking.

Ultrasonic Sensor Characteristics

- **Range:** Ultrasonic sensors typically have a range from a few centimeters to several meters (commonly 1-5 meters) depending on their application. In vehicle speed control, sensors with a range of 1-3 meters are often used to monitor close obstacles, while longer-range sensors might be used for traffic monitoring or adaptive cruise control.
- **Accuracy:** While ultrasonic sensors are very accurate at close ranges (with a tolerance

of a few centimeters), their effectiveness diminishes in very noisy environments or with irregularly shaped objects.

Integration in Speed Control

Ultrasonic sensors are usually integrated with other sensor systems, such as radar or cameras, to provide more reliable and comprehensive data for speed control decisions. The sensor data is processed by the vehicle's onboard computer, which then adjusts the vehicle's motor output (e.g., throttling back or braking) to ensure the vehicle maintains a safe speed based on its surroundings.

Example of Ultrasonic Sensor Use in Speed Control

- **Collision Prevention:** If an ultrasonic sensor detects an object ahead within a dangerous range, it could automatically reduce the vehicle's speed or engage the brakes to prevent a collision.
- **Automatic Braking:** In low-speed scenarios, such as in heavy traffic or parking lots, ultrasonic sensors help monitor the proximity of obstacles. The vehicle can automatically slow down or stop to avoid hitting an object.

Integration of Motors and Ultrasonic Sensors in Vehicle Speed Control

In an integrated vehicle speed control system, motors and ultrasonic sensors work together as follows:

- **Motor Adjustment Based on Sensor Input:** Ultrasonic sensors provide real-time data about the distance between the vehicle and obstacles. This data is processed by the onboard control system, which sends signals to the motor to adjust speed, such as reducing throttle or applying brakes to maintain a safe distance.
- **Adaptive Speed Regulation:** Motors adjust the vehicle's speed based on feedback from the ultrasonic sensors. For instance, if the sensors detect a vehicle ahead, the system could reduce the motor's power to slow down, ensuring a safe following distance.

Obstacle Avoidance: In environments with heavy traffic or congested roadways, the integration of motors and ultrasonic sensors helps the vehicle slow down or stop when obstacles are detected in the vehicle's path, ensuring safe operation.

Motor Control Techniques

- **PWM (Pulse Width Modulation):** This technique is used to control the power supplied to motors, which helps in adjusting their speed. By controlling the duty cycle of the PWM signal, the effective voltage reaching the motor is varied, thus regulating its speed.
- **Feedback Loop:** Many systems use feedback from a tachometer (a device that measures rotational speed) to monitor motor speed. If the vehicle is moving too fast or too slow, the control system adjusts the motor output accordingly.

CHAPTER 4

4.1 AVR MICROCONTROLLER

4.1.1 ARDUNIO:

The ARDUNIO ships as a bare circuit board with keypad, Lcd, and Gsm power supply

Introduction:

Arduino is a computer hardware and software company, project, and user community that designs and manufactures microcontroller kits for building digital devices and interactive objects that can sense and control objects in the physical world. The project's products are distributed as open-source hardware and software, which are licensed under the GNU Lesser General Public License (LGPL) or the GNU General Public License (GPL), permitting the manufacture of Arduino boards and software distribution by anyone. Arduino boards are available commercially in preassembled form, or as do-it-yourself kits.

Arduino board designs use a variety of microprocessors and controllers. The boards are equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards (*shields*) and other circuits. The boards feature serial communications interfaces, including Universal Serial Bus (USB) on some models, which are also used for loading programs from personal computers. The microcontrollers are typically programmed using a dialect of features from the programming languages C and C++. In addition to using traditional compiler toolchains, the Arduino project provides an integrated development environment (IDE) based on the Processing language project.

The Arduino project started in 2005 as a program for students at the Interaction Design Institute Ivrea in Ivrea, Italy,^[2] aiming to provide a low-cost and easy way for novices and professionals to create devices that interact with their environment using sensors and actuators. Common examples of such devices intended for beginner hobbyists include simple robots, thermostats, and motion detectors.

Despite its benefits, VSC faces challenges such as cybersecurity concerns, sensor reliability and accuracy, integration with existing infrastructure, public acceptance and adoption, and regulatory frameworks. Addressing these challenges will ensure widespread adoption of VSC technology.

The name Arduino comes from a bar in Ivrea, Italy, where some of the founders of the project used to meet. The bar was named after Arduin of Ivrea, who was the margrave of the March of Ivrea and King of Italy from 1002 to 1014.

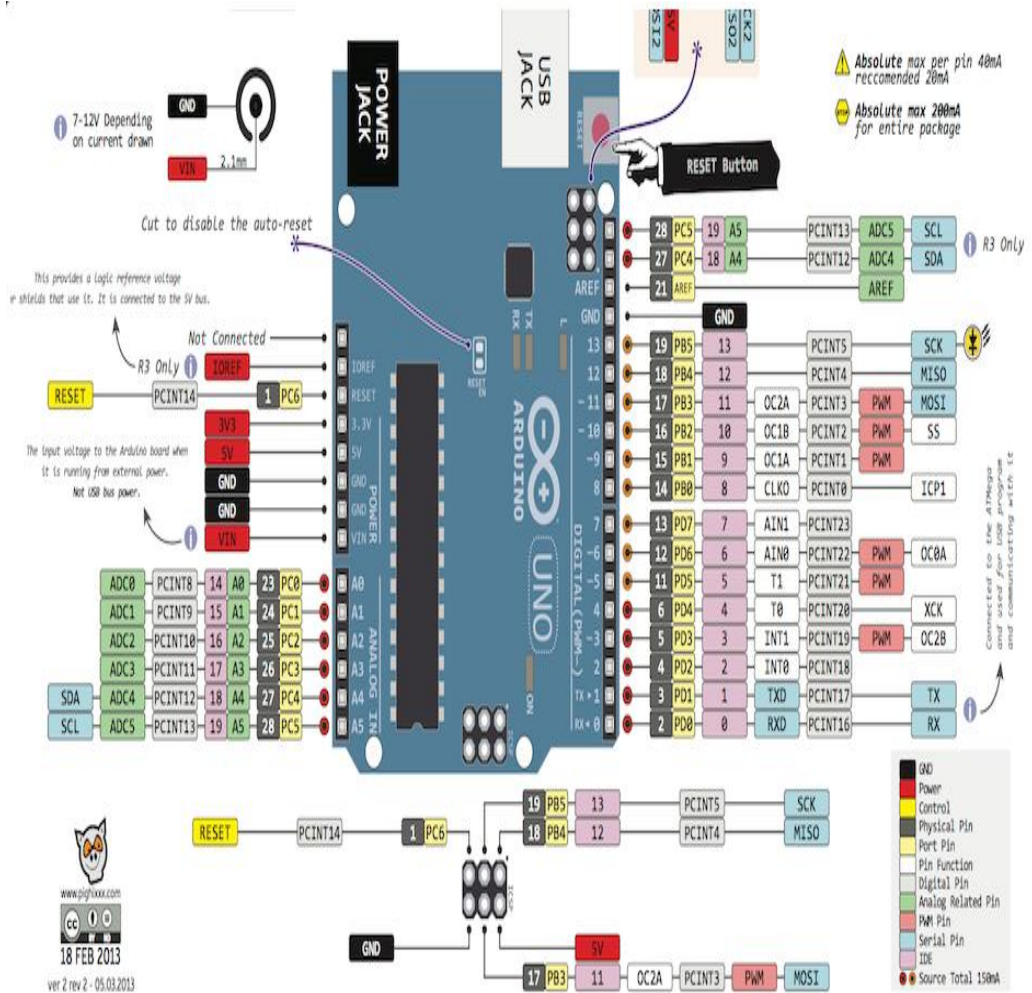


FIG-4.1. Arduino UNO

History:

The origin of the Arduino project started at the Interaction Design Institute Ivrea (IDII) in Ivrea, Italy.^[2] At that time, the students used a BASIC Stamp microcontroller at a cost of \$100, a considerable expense for many students. In 2004, Colombian student Hernando Barragán created the development platform *Wiring* as a Master's thesis project at IDII, under the supervision of Massimo Banzi and Casey Reas, who are known for work on the Processing language.

The project goal was to create simple, low cost tools for creating digital projects by

VEHICLE SPEED CONTROL SYSTEM
non-engineers. The Wiring platform consisted of a printed circuit board (PCB) with an ATmega168 microcontroller, an IDE based on Processing and library functions to easily program the microcontroller.

In 2005, Massimo Banzi, with David Mellis, another IDII student, and David Cuartielles, added support for the cheaper ATmega8 microcontroller to Wiring. But instead of continuing the work on Wiring, they copied the Wiring source code and renamed it as a separate project, called Arduino.

The boards feature serial communications interfaces, including Universal Serial Bus (USB) on some models, which are also used for loading programs from personal computers.

The boards use single-row pins or female headers that facilitate connections for programming and incorporation into other circuits. These may connect with add-on modules termed *shields*.

The initial Arduino core team consisted of Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino, and David Mellis, but Barragán was not invited to participate.

Following the completion of the Wiring platform, lighter and less-expensive versions were distributed in the open-source community.

Adafruit Industries, a New York City supplier of Arduino boards, parts, and assemblies, estimated in mid-2011 that over 300,000 official Arduinos had been commercially produced, and in 2013 that 700,000 official boards were in users' hands.

4.2 Hardware:

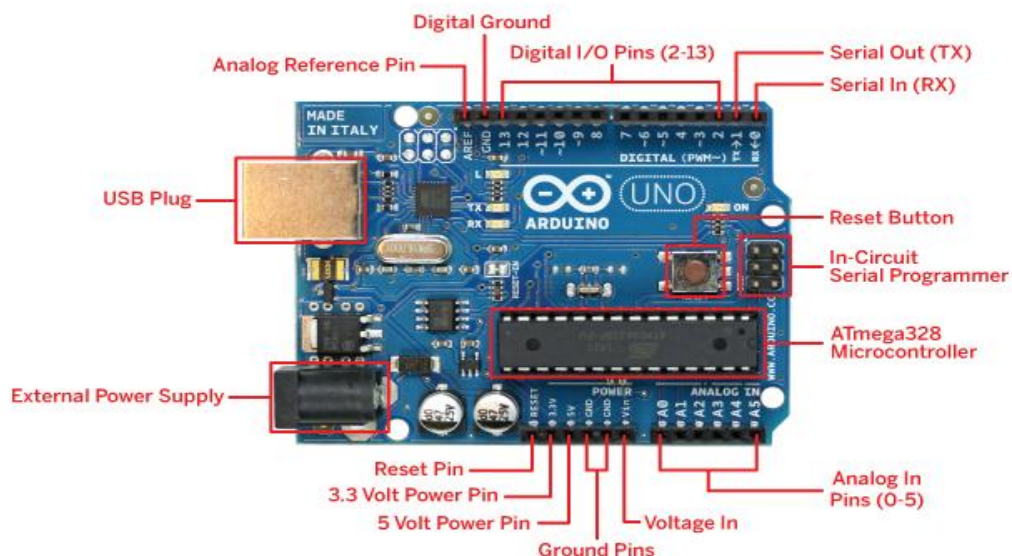


FIG-4.2. Arduino Hardware

Arduino is open-source hardware. The hardware reference designs are distributed under a Creative Commons Attribution Share-Alike 2.5 license and are available on the Arduino website. Layout and production files for some versions of the hardware are also available. The source code for the IDE is released under the GNU General Public License, version 2. Nevertheless, an official Bill of Materials of Arduino boards has never been released by Arduino staff.

Although the hardware and software designs are freely available under copyleft licenses, the developers have requested that the name *Arduino* be exclusive to the official product and not be used for derived works without permission. The official policy document on use of the Arduino name emphasizes that the project is open to incorporating work by others into the official product.^[9] Several Arduino-compatible products commercially released have avoided the project name by using various names ending in - *arduino*.

An Arduino board consists of an Atmel 8-16- or 32-bit AVR microcontroller (ATmega8, ATmega168, ATmega328, ATmega1280, ATmega2560), but other makers' microcontrollers have been used since 2015. The boards use single-row pins or female headers that facilitate connections for programming and incorporation into other circuits. These may connect with add-on modules termed *shields*. Multiple, and possibly stacked shields may be individually addressable via an I²C serial bus. Most boards include a 5 V linear regulator and a 16 MHz crystal oscillator or ceramic resonator. Some designs, such as the LilyPad, run at 8 MHz and dispense with the onboard voltage regulator due to specific form-factor restrictions.

Arduino microcontrollers are pre-programmed with a boot loader that simplifies uploading of programs to the on-chip flash memory. The default bootloader of the Arduino is the optibootbootloader. Boards are loaded with program code via a serial connection to another computer. Some serial Arduino boards contain a level shifter circuit to convert between RS-232 logic levels and transistor–transistor logic (TTL) level signals. Current Arduino boards are programmed via Universal Serial Bus (USB), implemented using USB-to-serial adapter chips such as the FTDI FT232.

Some boards, such as later-model Uno boards, substitute the FTDI chip with a separate AVR chip containing USB-to-serial firmware, which is reprogrammable via its own ICSP header. Other variants, such as the Arduino Mini and the unofficial

arduino, use a detachable USB-to-serial adapter board or cable, Bluetooth or other methods, when used with traditional microcontroller tools instead of the ArduinoIDE, standard AVR in-system programming (ISP) programming is used.

The Arduino board exposes most of the microcontroller's I/O pins for use by other circuits. The *Diecimila*, *Duemilanove*, and current *Uno*¹ provide 14 digital I/O pins, six of which can produce pulse-width modulated signals, and six analog inputs, which can also be used as six digital I/O pins. These pins are on the top of the board, via female 0.1-inch (2.54 mm) headers. Several plug-in application shields are also commercially available. The ArduinoNano, and Arduino-compatible Bare Bones Board^[13] and Boarduino boards may provide male header pins on the underside of the board that can plug into solderless breadboards.

Many Arduino-compatible and Arduino-derived boards exist. Some are functionally equivalent to an Arduino and can be used interchangeably. Many enhance the basic Arduino by adding output drivers, often for use in school-level education, to simplify making buggies and small robots. Others are electrically equivalent but change the form factor, sometimes retaining compatibility with shields, sometimes not. Some variants use different processors, of varying compatibility.

Official boards

The original Arduino hardware was produced by the Italian company Smart Projects.^[15] Some Arduino-branded boards have been designed by the American companies SparkFun Electronics and Adafruit Industries. As of 2016, 17 versions of the Arduino hardware have been commercially produced.

Software development

A program for Arduino may be written in any programming language for a compiler that produces binary machine code for the target processor. Atmel provides a development environment for their microcontrollers, AVR Studio and the newer Atmel Studio.

The Arduino project provides the Arduino integrated development environment (IDE), which is a cross-platform application written in the programming language Java. It originated from the IDE for the languages *Processing* and *Wiring*. It includes a code editor with features such as text cutting and pasting, searching and replacing text, automatic indenting, brace matching, and syntax highlighting, and provides simple *one-click* mechanisms to compile and upload programs to an Arduino board. It also contains a message area, a text console, a toolbar with buttons for common

functions and a hierarchy of operation menus.

A program written with the IDE for Arduino is called a *sketch*.^[40] Sketches are saved on the development computer as text files with the file extension *.ino*. Arduino Software (IDE) pre-1.0 saved sketches with the extension *.pde*.

The Arduino IDE supports the languages C and C++ using special rules of code structuring. The Arduino IDE supplies a software library from the Wiring project, which provides many common input and output procedures. User-written code only requires two basic functions, for starting the sketch and the main program loop, that are compiled and linked with a program stub *main()* into an executable cyclic executive program with the GNU toolchain, also included with the IDE distribution. The Arduino IDE employs the program *avrdude* to convert the executable code into a text file in hexadecimal encoding that is loaded into the Arduino board by a loader program in the board's firmware.

Applications

- Xoscillo, an open-source oscilloscope^[48]
- Arduinome, a MIDI controller device that mimics the Monome
- OBDuino, a trip computer that uses the on-board diagnostics interface found in most modern cars
- Ardupilot, drone software and hardware
- Gameduino, an Arduino shield to create retro 2D video games^[49]
- ArduinoPhone, a do-it-yourself cellphone^{[50][51]}
- Water quality testing platform
- Automatic titration system based on Arduino and stepper motor^[53]
- Low cost data glove for virtual reality applications^[54]
- Impedance sensor system to detect bovine milk adulteration^[55]
- Homemade CNC using Arduino and DC motors with close loop control by Homofaciens
- DC motor control using Arduino and H-Bridge^[56]

Technical specs

Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
PWM Digital I/O Pins	6
Analog Input Pins	6
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega328P) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)
Clock Speed	16 MHz
LED_BUILTIN	13
Length	68.6 mm
Width	53.4 mm
Weight	25 g

Table –4.1 Components and their range

The Arduino/Genuino Uno can be programmed with the (Arduino Software (IDE)). Select "Arduino/Genuino Uno from the Tools > Board menu (according to the microcontroller on your board). For details, see the reference and tutorials.

The ATmega328 on the Arduino/Genuino Uno comes preprogrammed with a bootloader that allows you to upload new code to it without the use of an external hardware programmer. It communicates using the original STK500 protocol (reference, C header files).

You can also bypass the bootloader and program the microcontroller through the ICSP (In-Circuit Serial Programming) header using Arduino ISP or similar; see these instructions for details.

The ATmega16U2 (or 8U2 in the rev1 and rev2 boards) firmware source code is available in the Arduino repository. The ATmega16U2/8U2 is loaded with a DFU bootloader, which can be activated by:

- **On Rev1 boards:** connecting the solder jumper on the back of the board (near the map of Italy) and then resealing the 8U2.
- **On Rev2 or later boards:** there is a resistor that pulling the 8U2/16U2 HWB line to ground, making it easier to put into DFU mode.

You can then use Atmel's FLIP software (Windows) or the DFU programmer (Mac OS X and Linux) to load a new firmware. Or you can use the ISP header with an external programmer (overwriting the DFU bootloader). See this user-contributed tutorial for more information.

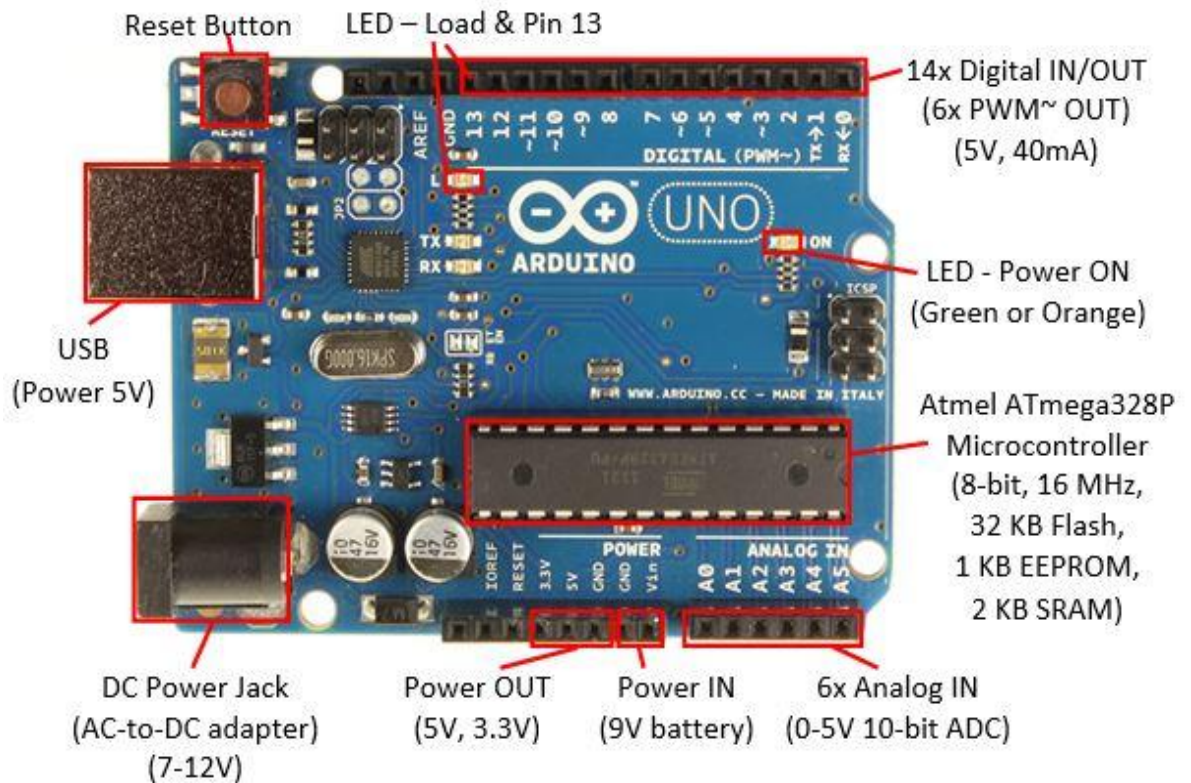


FIG-4.3 Arduino Connections

Atmega168 Pin Mapping

Arduino function				Arduino function
reset	(PCINT14/RESET) PC6	1	28	PC5 (ADC5/SCL/PCINT13) analog input 5
digital pin 0 (RX)	(PCINT16/RXD) PD0	2	27	PC4 (ADC4/SDA/PCINT12) analog input 4
digital pin 1 (TX)	(PCINT17/TXD) PD1	3	26	PC3 (ADC3/PCINT11) analog input 3
digital pin 2	(PCINT18/INT0) PD2	4	25	PC2 (ADC2/PCINT10) analog input 2
digital pin 3 (PWM)	(PCINT19/OC2B/INT1) PD3	5	24	PC1 (ADC1/PCINT9) analog input 1
digital pin 4	(PCINT20/XCK/T0) PD4	6	23	PC0 (ADC0/PCINT8) analog input 0
VCC	VCC	7	22	GND GND
GND	GND	8	21	AREF analog reference
crystal	(PCINT6/XTAL1/TOSC1) PB6	9	20	AVCC VCC
crystal	(PCINT7/XTAL2/TOSC2) PB7	10	19	PB5 (SCK/PCINT5) digital pin 13
digital pin 5 (PWM)	(PCINT21/OC0B/T1) PD5	11	18	PB4 (MISO/PCINT4) digital pin 12
digital pin 6 (PWM)	(PCINT22/OC0A/AIN0) PD6	12	17	PB3 (MOSI/OC2A/PCINT3) digital pin 11 (PWM)
digital pin 7	(PCINT23/AIN1) PD7	13	16	PB2 (SS/OC1B/PCINT2) digital pin 10 (PWM)
digital pin 8	(PCINT0/CLKO/ICP1) PB0	14	15	PB1 (OC1A/PCINT1) digital pin 9 (PWM)

Digital Pins 11, 12 & 13 are used by the ICSP header for MOSI, MISO, SCK connections (Atmega168 pins 17, 18 & 19). Avoid low-impedance loads on these pins when using the ICSP header.

FIG-4.4. Atmega 168 pin mapping

The Arduino/Genuino Uno has a resettable polyfuse that protects your computer's USB ports from shorts and overcurrent. Although most computers provide their own internal protection, the fuse provides an extra layer of protection. If more than 500 mA is applied to the USB port, the fuse will automatically break the connection until the short or overload is removed.

The Uno differs from all preceding boards in that it does not use the FTDI USB-to-serial driver chip. Instead, it features the Atmega16U2 (Atmega8U2 up to version

R2) programmed as a USB-to-serial converter.

Power

The Arduino/Genuino Uno board can be powered via the USB connection or with an external power supply. The power source is selected automatically.

External (non-USB) power can come either from an AC-to-DC adapter (wall-wart) or battery. The adapter can be connected by plugging a 2.1mm center-positive plug into the board's power jack. Leads from a battery can be inserted in the GND and Vin pin headers of the POWER connector.040 48576663

The board can operate on an external supply from 6 to 20 volts. If supplied with less than 7V, however, the 5V pin may supply less than five volts and the board may become unstable. If using more than 12V, the voltage regulator may overheat and damage the board. The recommended range is 7 to 12 volts.

The power pins are as follows:

- Vin. The input voltage to the Arduino/Genuino board when it's using an external power source (as opposed to 5 volts from the USB connection or other regulated power source). You can supply voltage through this pin, or, if supplying voltage via the power jack, access it through this pin.
- 5V. This pin outputs a regulated 5V from the regulator on the board. The board can be supplied with power either from the DC power jack (7 - 12V), the USB connector (5V), or the VIN pin of the board (7-12V). Supplying voltage via the 5V or 3.3V pins bypasses the regulator, and can damage your board. We don't advise it.
- 3V3. A 3.3 volt supply generated by the on-board regulator. Maximum current draw is 50 mA.
- GND. Ground pins.
- IOREF. This pin on the Arduino/Genuino board provides the voltage reference with which the microcontroller operates. A properly configured shield can read the IOREF pin voltage and select the appropriate power source or enable voltage translators on the outputs to work with the 5V or 3.3V.

Memory

The ATmega328 has 32 KB (with 0.5 KB occupied by the bootloader). It also has 2 KB of SRAM and 1 KB of EEPROM (which can be read and written with the EEPROM library).

Input and Output

See the mapping between Arduino pins and ATmega328P ports. The mapping for the Atmega8, 168, and 328 is identical.

Each of the 14 digital pins on the Uno can be used as an input or output, using `pinMode()`, `digitalWrite()`, and `digitalRead()` functions. They operate at 5 volts. Each pin can provide or receive 20 mA as recommended operating condition and has an internal pull-up resistor (disconnected by default) of 20-50k ohm. A maximum of 40mA is the value that must not be exceeded on any I/O pin to avoid permanent damage to the microcontroller.

In addition, some pins have specialized functions:

- **Serial:** 0 (RX) and 1 (TX). Used to receive (RX) and transmit (TX) TTL serial data. These pins are connected to the corresponding pins of the ATmega8U2 USB-to-TTL Serial chip.
- **External Interrupts:** 2 and 3. These pins can be configured to trigger an interrupt on a low value, a rising or falling edge, or a change in value. See the `attachInterrupt()` function for details.
- **PWM:** 3, 5, 6, 9, 10, and 11. Provide 8-bit PWM output with the `analogWrite()` function.
- **SPI:** 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK). These pins support SPI communication using the SPI library.
- **LED:** 13. There is a built-in LED driven by digital pin 13. When the pin is HIGH value, the LED is on, when the pin is LOW, it's off.
- **TWI:** A4 or SDA pin and A5 or SCL pin. Support TWI communication using the Wire library.

The Uno has 6 analog inputs, labeled A0 through A5, each of which provide 10 bits of resolution (i.e. 1024 different values). By default they measure from ground to 5 volts, though is it possible to change the upper end of their range using the AREF pin and the `analogReference()` function.

There are a couple of other pins on the board:

- **AREF.** Reference voltage for the analog inputs. Used with `analogReference()`.
- **Reset.** Bring this line LOW to reset the microcontroller. Typically used to add a reset button to shields which block the one on the board.

Communication

Arduino/Genuino Uno has a number of facilities for communicating with a computer, another Arduino/Genuino board, or other microcontrollers. The ATmega328 provides UART TTL (5V) serial communication, which is available on digital pins 0 (RX) and 1 (TX). An ATmega16U2 on the board channels this serial communication over USB and appears as a virtual com port to software on the computer. The 16U2 firmware uses the standard USB COM drivers, and no external driver is needed. However, on Windows, a .inf file is required. The Arduino Software (IDE) includes a serial monitor which allows simple textual data to be sent to and from the board. The RX and TX LEDs on the board will flash when data is being transmitted via the USB-to-serial chip and USB connection to the computer (but not for serial communication on pins 0 and 1).

A SoftwareSerial library allows serial communication on any of the Uno's digital pins.

The ATmega328 also supports I2C (TWI) and SPI communication. The Arduino Software (IDE) includes a Wire library to simplify use of the I2C bus; see the documentation for details. For SPI communication, use the SPI library.

Automatic (Software) Reset Rather than requiring a physical press of the reset button before an upload, the Arduino/Genuino Uno board is designed in a way that allows it to be reset by software running on a connected computer. One of the hardware flow control lines (DTR) of the ATmega8U2/16U2 is connected to the reset line of the ATmega328 via a 100 nanofarad capacitor. When this line is asserted (taken low), the reset line drops long enough to reset the chip. The Arduino Software (IDE) uses this capability to allow you to upload code by simply pressing the upload button in the interface toolbar. This means that the bootloader can have a shorter timeout, as the lowering of DTR can be well-coordinated with the start of the upload.

This setup has other implications. When the Uno is connected to either a computer running Mac OS X or Linux, it resets each time a connection is made to it from software (via USB). For the following half-second or so, the bootloader is running on the Uno. While it is programmed to ignore malformed data (i.e. anything besides an upload of new code), it will intercept the first few bytes of data sent to the board after a connection is opened. If a sketch running on the board receives one-time configuration or other data when it first starts, make sure that the software with which it communicates waits a second after opening the connection and before sending this data.

An ATmega16U2 on the board channels this serial communication over USB and appears as a virtual com port to software on the computer. The 16U2 firmware uses the standard USB COM drivers, and no external driver is needed. However, on Windows, a .inf file is required. The Arduino Software (IDE) includes a serial monitor which allows simple textual data to be sent to and from the board.

The Uno board contains a trace that can be cut to disable the auto-reset. The pads on either side of the trace can be soldered together to re-enable it. It's labeled "RESET-EN". You may also be able to disable the auto-reset by connecting a 110 ohm resistor from 5V to the reset line; see this forum thread for details.

Revisions

Revision 3 of the board has the following new features:

- 1.0 pinout: added SDA and SCL pins that are near to the AREF pin and two other new pins placed near to the RESET pin, the IOREF that allow the shields to adapt to the voltage provided from the board. In future, shields will be compatible with both the board that uses the AVR, which operates with 5V and with the Arduino Due that operates with 3.3V. The second one is a not connected pin, that is reserved for future purposes.
- Stronger RESET circuit.
- Atmega 16U2 replace the 8U2.

CHAPTER 5

FUNCTIONAL MODULES

5.1 Power Supply:

The input to the circuit is applied from the regulated power supply. The a.c. input i.e., 230V from the mains supply is step down by the transformer to 12V and is fed to a rectifier. The output obtained from the rectifier is a pulsating d.c voltage. So in order to get a pure d.c voltage, the output voltage from the rectifier is fed to a filter to remove any a.c components present even after rectification. Now, this voltage is given to a voltage regulator to obtain a pure constant dc voltage. The block diagram of regulated power supply is shown in the figure 3.2

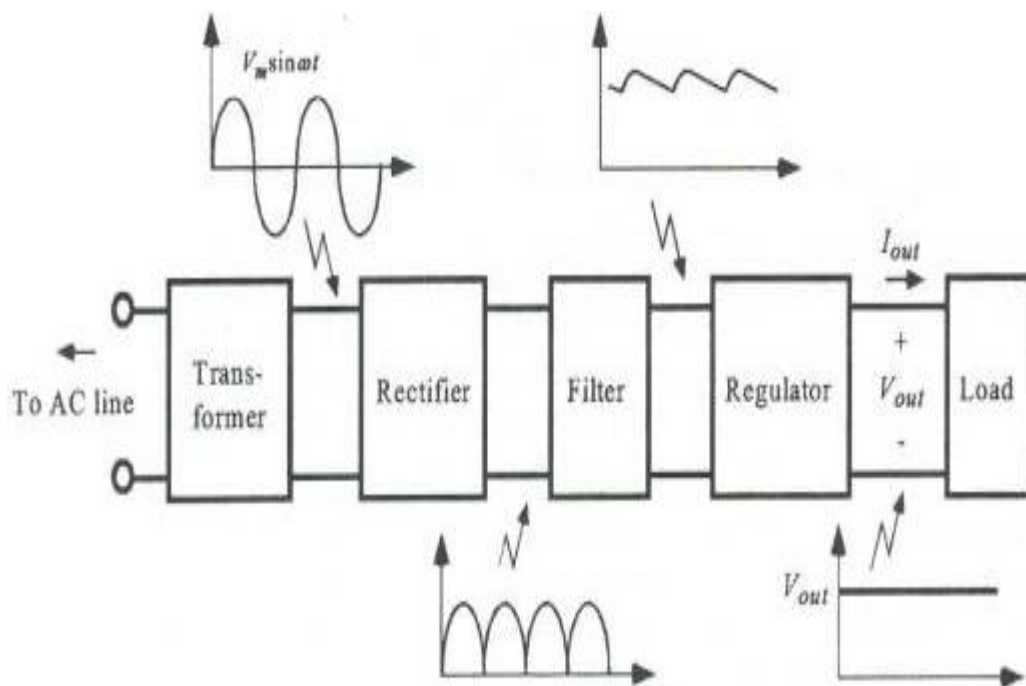


Fig 5.1 components of power supply

Transformer:

Usually, DC voltages are required to operate various electronic equipment and these voltages are 5V, 9V or 12V. But these voltages cannot be obtained directly. Thus the a.c input available at the mains supply i.e., 230V is to be brought down to

the required voltage level. This is done by a transformer. Thus, a step down transformer is employed to decrease the voltage to a required level.

Rectifier:

The output from the transformer is fed to the rectifier. It converts A.C. into pulsating D.C. The rectifier may be a half wave or a full wave rectifier. In this project, a bridge rectifier is used because of its merits like good stability and full wave rectification.

Filter:

Capacitive filter is used in this project. It removes the ripples from the output of rectifier and smoothens the D.C. Output received from this filter is constant until the mains voltage and load is maintained constant. However, if either of the two is varied, D.C. voltage received at this point changes. Therefore a regulator is applied at the output stage.

Voltage regulator:

As the name itself implies, it regulates the input applied to it. A voltage regulator is an electrical regulator designed to automatically maintain a constant voltage level. In this project, power supply of 5V and 12V are required. In order to obtain these voltage levels, 7805 and 7812 voltage regulators are to be used. The first number 78 represents positive supply and the numbers 05, 12 represent the required output voltage

LCD

LCD (Liquid Crystal Display) screen is an electronic display module and find a wide range of applications. A 16x2 LCD display is very basic module and is very commonly used in various devices and circuits. These modules are preferred over seven segments and other multi segment LEDs. The reasons being: LCDs are economical; easily programmable; have no limitation of displaying special & even

custom characters (unlike in seven segments), animations and so on.

A **16x2 LCD** means it can display 16 characters per line and there are 2 such lines. In this LCD each character is displayed in 5x7 pixel matrix. This LCD has two registers, namely, Command and Data. The command register stores the command instructions given to the LCD. A command is an instruction given to LCD to do a predefined task like initializing it, clearing its screen, setting the cursor position, controlling display etc. The data register stores the data to be displayed on the LCD. The data is the ASCII value of the character to be displayed on the LCD.

LOAD

of the **Vehicle Speed Control**, the term "load" can be understood as the various operational, environmental, and system-related factors that the speed control system must handle to function efficiently. These factors influence the performance, stability, and safety of the vehicle, and the system must dynamically adjust its operations to accommodate these changing loads. The load in the vehicle speed control system can be divided into several categories, each impacting the vehicle's operation in different ways.

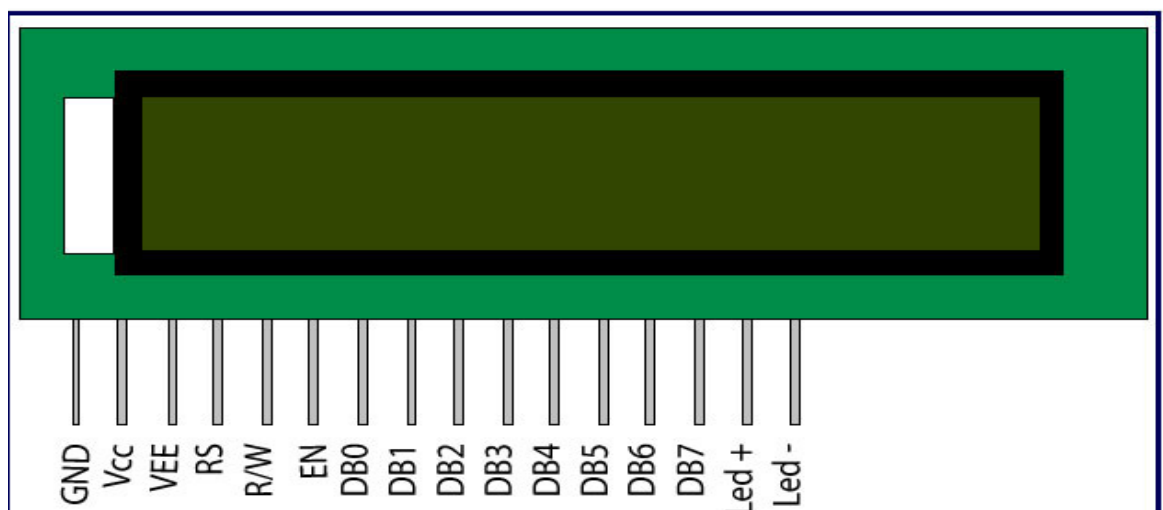


FIG 5.2 LED Display

In a brushed DC motor, the brushes make mechanical contact with a set of

electrical contacts provided on a commutation secured to an armature, forming an electrical circuit between the DC electrical source and coil windings on the armature. As the armature rotates on an axis, the stationary brushes come into contact with different sections of the rotating commutator.

Permanent magnet DC motors utilize two or more brushes contacting a commutator which provides the direct current flow to the windings of the rotor, which in turn provide the desired magnetic repulsion/attraction with the permanent magnets located around the periphery of the motor.

Pin No	Function	Name
1	Ground (0V)	Ground
2	Supply voltage; 5V (4.7V – 5.3V)	V _{cc}
3	Contrast adjustment; through a variable resistor	V _{EE}
4	Selects command register when low; and data register when high	Register Select
5	Low to write to the register; High to read from the register	Read/write
6	Sends data to data pins when a high to low pulse is given	Enable
7	8-bit data pins	DB0
8		DB1
9		DB2
10		DB3
11		DB4
12		DB5
13		DB6
14		DB7
15	Backlight V _{CC} (5V)	Led+
16	Backlight Ground (0V)	Led-

Table –5.1 Functions of Pin diagram

DC MOTOR (Electric Motor)

An **electric motor** is an electromechanical device that converts electrical energy into mechanical energy.

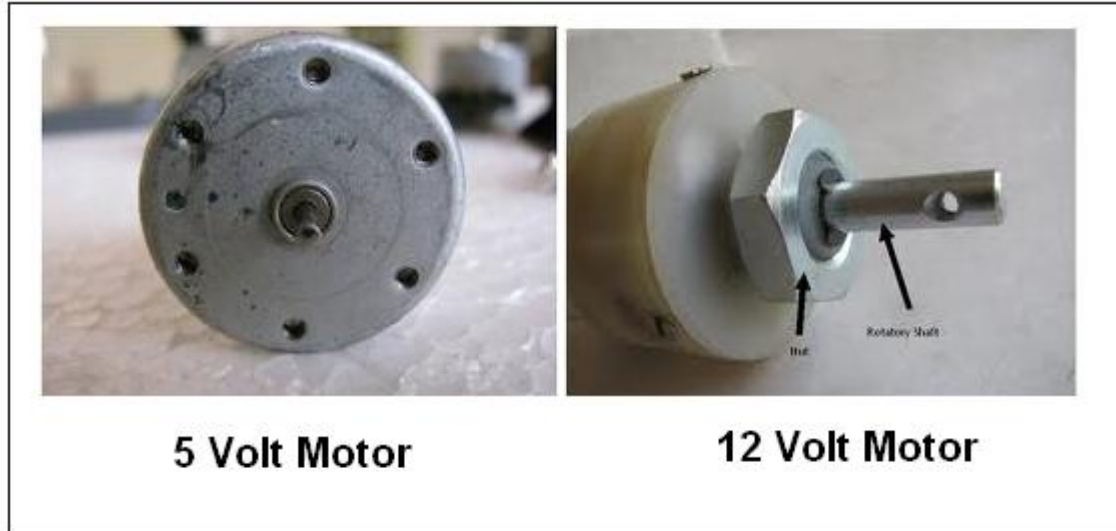


FIG-5.3 DC Motors

Most electric motors operate through the interaction of magnetic fields and current-carrying conductors to generate force. The reverse process, producing electrical energy from mechanical energy, is done by generators such as an alternator or a dynamo; some electric motors can also be used as generators, for example, a traction motor on a vehicle may perform both tasks. Electric motors and generators are commonly referred to as electric machines.

Electric motors are found in applications as diverse as industrial fans, blowers and pumps, machine tools, household appliances, power tools, and disk drives.

DC Motor Working:

Direct current (DC) motors are widely used to generate motion in a variety of products. Permanent magnet DC (direct current) motors are enjoying increasing popularity in applications requiring compact size, high torque, high efficiency, and low power consumption.

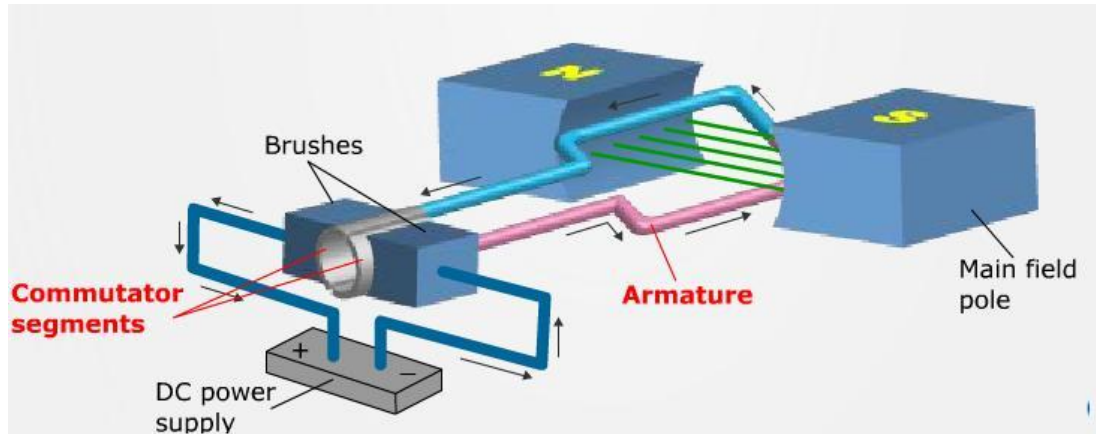


FIG-5.4 DC Motor working

In a brushed DC motor, the brushes make mechanical contact with a set of electrical contacts provided on a commutation secured to an armature, forming an electrical circuit between the DC electrical source and coil windings on the armature. As the armature rotates on an axis, the stationary brushes come into contact with different sections of the rotating commutator.

Permanent magnet DC motors utilize two or more brushes contacting a commutator which provides the direct current flow to the windings of the rotor, which in turn provide the desired magnetic repulsion/attraction with the permanent magnets located around the periphery of the motor. The brushes are conventionally located in brush boxes and utilize a U-shaped spring which biases the brush into contact with the commutator. Permanent magnet brushless dc motors are widely used in a variety of applications due to their simplicity of design, high efficiency, and low noise. These motors operate by electronic commutation of stator windings rather than the conventional mechanical commutation accomplished by the pressing engagement of brushes against a rotating commutator.

A brushless DC motor basically consists of a shaft, a rotor assembly equipped with one or more permanent magnets arranged on the shaft, and a stator assembly which incorporates a stator component and phase windings. Rotating magnetic fields are formed by the currents applied to the coils. The rotator is formed of at least one permanent magnet surrounded by the stator, wherein the rotator rotates within the stator. Two bearings are mounted at an axial distance to each other on the shaft to

VEHICLE SPEED CONTROL SYSTEM

support the rotor assembly and stator assembly relative to each other. To achieve electronic commutation, brushless dc motor designs usually include an electronic controller for controlling the excitation of the stator windings.

INTRODUCTION TO L293d

INTRODUCTION:

- 600mA OUTPUT CURRENT CAPABILITY PER CHANNEL
- 1.2A PEAK OUTPUT CURRENT (non repetitive) PER CHANNEL
- ENABLE FACILITY
- OVERTEMPERATURE PROTECTION
- LOGICAL "0" INPUT VOLTAGE UP TO 1.5 V
(HIGH NOISE IMMUNITY)
- INTERNAL CLAMP DIODES

5.2 DESCRIPTION

The Device is a monolithic integrated high voltage, high current four channel driver designed to accept standard DTL or TTL logic levels and drive inductive loads (such as relays solenoids, DC and stepping motors) and switching power transistors. To simplify use as two bridges each pair of channels is equipped with an enable input. A separate supply input is provided for the logic, allowing operation at a lower voltage and internal clamp diodes are included. This device is suitable for use in switching applications at frequencies up to 5 kHz. The L293D is assembled in a 16 lead plastic package which has 4 center pins connected together and used for heat sinking. The L293DD is assembled in a 20 lead surface mount which has 8 center pins connected together and used for heat sinking. L293D is a dual *H-bridge* motor driver integrated circuit (IC). Motor drivers act as current amplifiers since they take a low-current control signal and provide a higher-current signal. This higher current signal is used to drive the motors. Logic 01 and 10 will rotate it in clockwise and anticlockwise directions, respectively. Enable pins 1 and 9 (corresponding to the two motors) must be high for motors to start operating. When an enable input is high, the associated driver gets enabled. As a result, the outputs become active and work in phase with their inputs.

L293D contains two inbuilt H-bridge driver circuits. In its common mode of operation, two DC motors can be driven simultaneously, both in forward and reverse direction. The motor operations of two motors can be controlled by input logic at pins 2 & 7 and 10 & 15. Input logic 00 or 11 will stop the corresponding motor. Logic 01 and 10 will rotate it in clockwise and anticlockwise directions, respectively. Enable pins 1 and 9 (corresponding to the two motors) must be high for motors to start operating. When an enable input is high, the associated driver gets enabled. As a result, the outputs become active and work in phase with their inputs. Similarly, when the enable input is low, that driver is disabled, and their outputs are off and in the high-impedance state.

PIN DESCRIPTION:

Pin No	Function	Name
1	Enable pin for Motor 1; active high	Enable 1,2
2	Input 1 for Motor 1	Input 1
3	Output 1 for Motor 1	Output 1
4	Ground (0V)	Ground
5	Ground (0V)	Ground
6	Output 2 for Motor 1	Output 2
7	Input 2 for Motor 1	Input 2
8	Supply voltage for Motors; 9-12V (up to 36V)	Vcc ₂
9	Enable pin for Motor 2; active high	Enable 3,4
10	Input 1 for Motor 1	Input 3
11	Output 1 for Motor 1	Output 3
12	Ground (0V)	Ground
13	Ground (0V)	Ground
14	Output 2 for Motor 1	Output 4
15	Input2 for Motor 1	Input 4
16	Supply voltage; 5V (up to 36V)	

Table –5.2 Pin Description

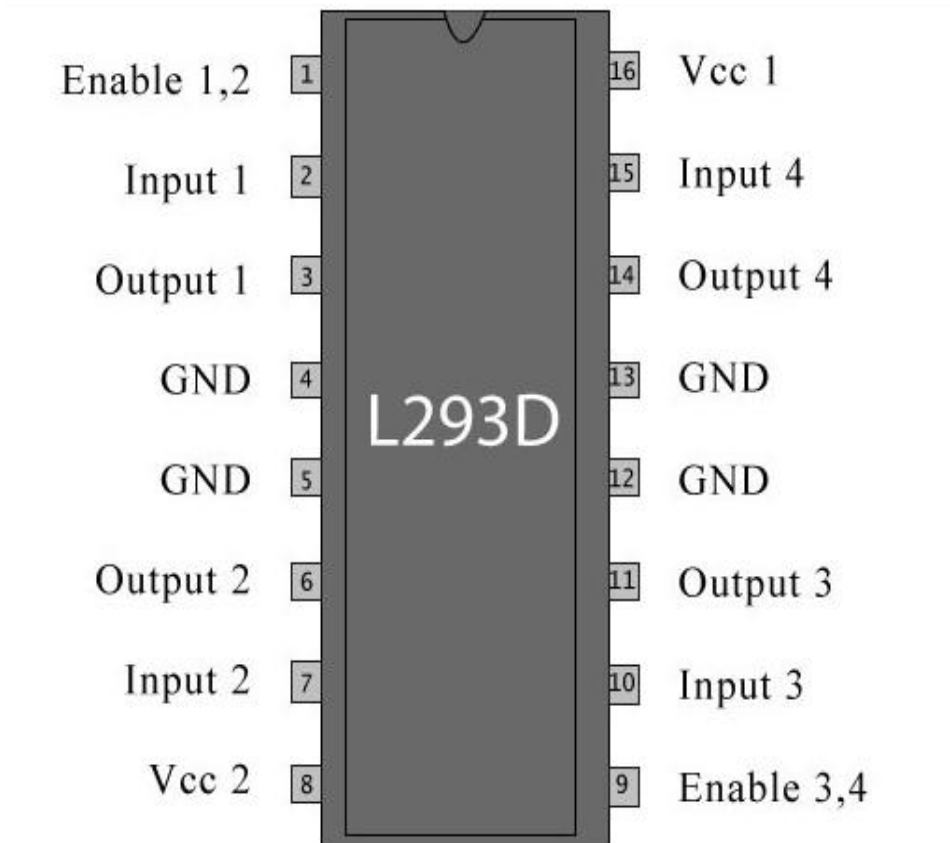
PIN DIAGRAM:

FIG-5.5 Pin diagram of L293D

EXPLANATION OF EACH PINS**Power and Ground Pins**

- 1. VCC (Power):** Supplies power to the Electronic Control Unit (ECU) or speed control module.
- 2. GND (Ground):** Provides a ground connection for the ECU or speed control module

3. Sensor Input Pins

- 1. VS (Vehicle Speed):** Receives the vehicle speed signal from the speed sensor.
- 2. BP (Brake Pedal):** Monitors the brake pedal position or switch status.
- 3. AS (Accelerator Pedal):** Tracks the accelerator pedal position or switch status.

4. Actuator Output Pins

- 1. OA (Output A):** Controls the speed control actuator, such as a throttle valve or brake actuator.

2. OB (Output B): Optional output for additional actuators or accessories.

3. Communication Pins

1. CANH (CAN High): Transmits data to other vehicle systems via the Controller Area Network (CAN) bus.

2. CANL (CAN Low): Receives data from other vehicle systems via the CAN bus.

3. Other Pins

4. 1. RST (Reset): Resets the ECU or speed control module to its default state.

5. 2. DI (Diagnostic Input): Allows for diagnostic testing and troubleshooting of the speed control system.

Please note that the specific pins and their functions may vary depending on the vehicle make, model, and speed control system design.

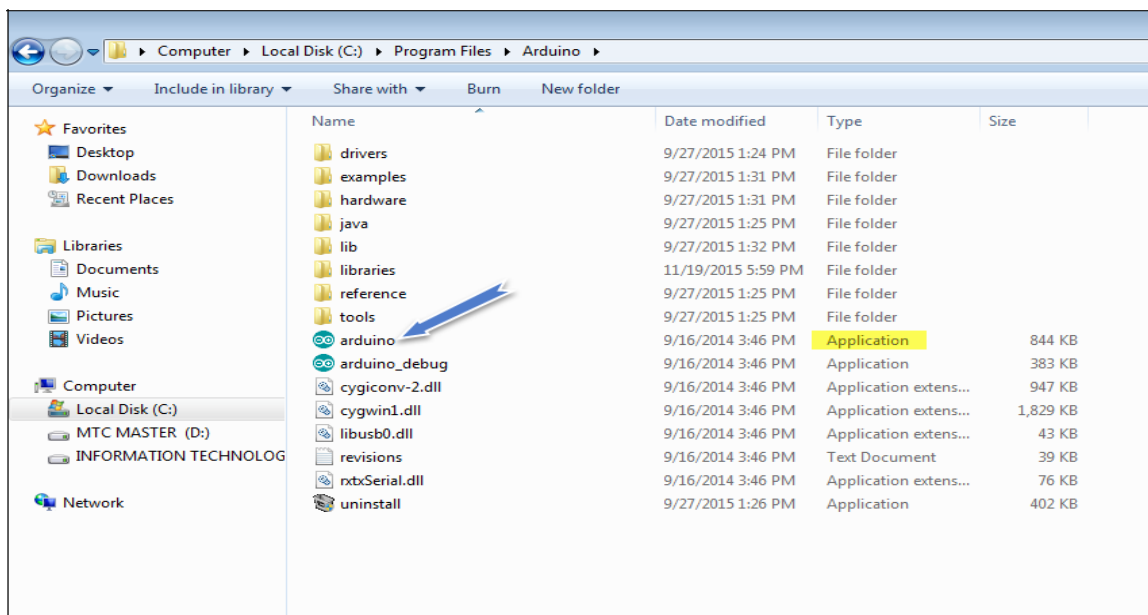
CHAPTER 6

SOFTWARE MODULES DESCRIPTION

6.11 ARDUNIO INSTALLATION:

After learning about the main parts of the Arduino UNO board, we are ready to learn how to set up the Arduino IDE. Once we learn this, we will be ready to upload our program on the Arduino board.

In this section, we will learn in easy steps, how to set up the Arduino IDE on our

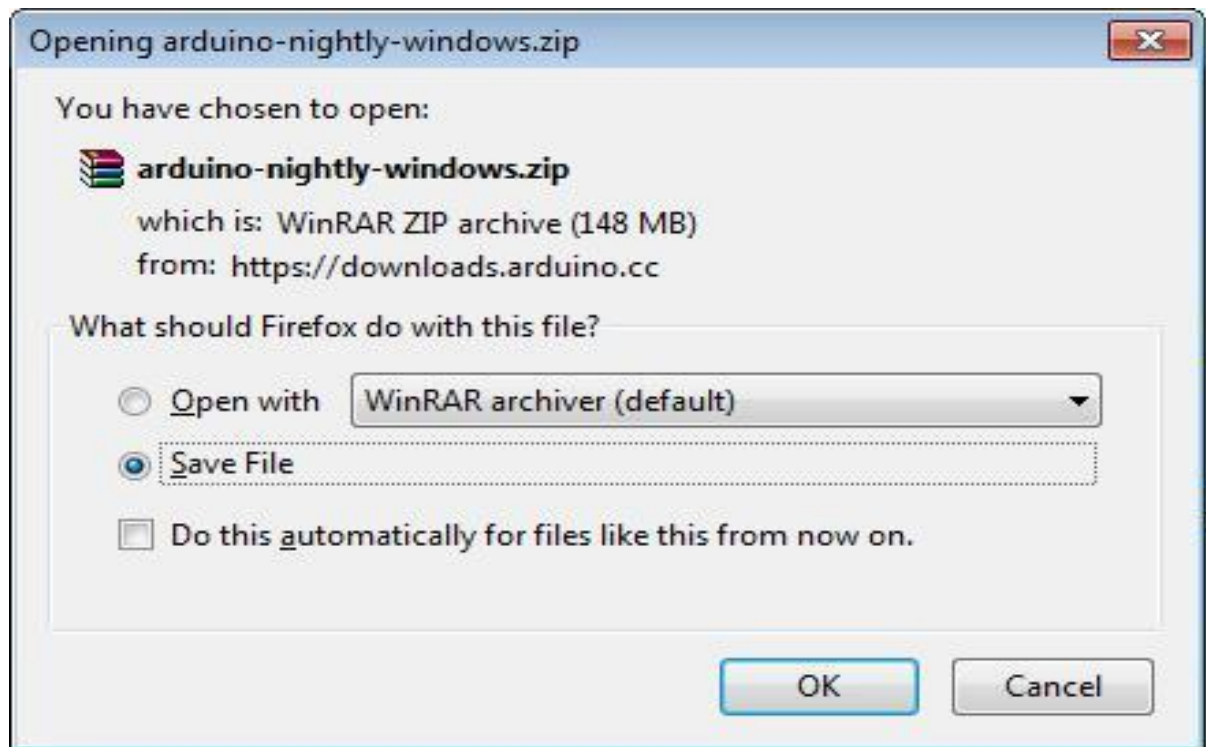


computer and prepare the board to receive the program via USB cable.

Step 1: First you must have your Arduino board (you can choose your favorite board) and a USB cable. In case you use Arduino UNO, Arduino Duemilanove, Nano, Arduino Mega 2560, or Diecimila, you will need a standard USB cable (A plug to B plug), the kind you would connect to a USB printer as shown in the following image.

Step 2: Download Arduino IDE Software.

You can get different versions of Arduino IDE from the Download page on the Arduino Official website. You must select your software, which is compatible with your operating system (Windows, IOS, or Linux). After your file download is complete, unzip the file.



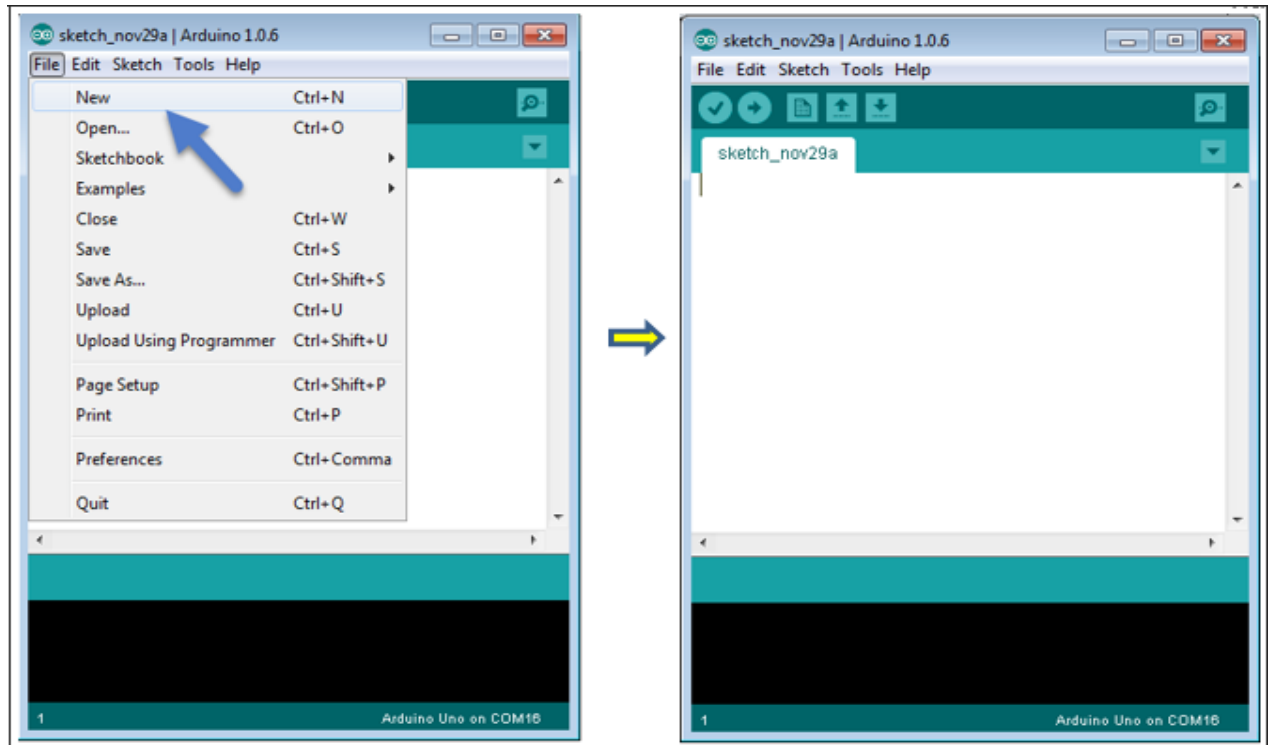
Step 3: Power up your board.

The Arduino Uno, Mega, Duemilanove and ArduinoNano automatically draw power from either, the USB connection to the computer or an external power supply. If you are using an ArduinoDiecimila, you have to make sure that the board is configured to draw power from the USB connection. The power source is selected with a jumper, a small piece of plastic that fits onto two of the three pins between the USB and power jacks. Check that it is on the two pins closest to the USB port.

Connect the Arduino board to your computer using the USB cable. The green power LED (labeled PWR) should glow.

Step 4: Launch Arduino IDE.

After your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Double-click the icon to start the IDE.



Step 5: Open your first project.

Once the software starts, you have two options:

Create a new project.

Open an existing project example.

To create a new project, select File --> New

To open an existing project example, select File -> Example -> Basics -> Blink.

Here, we are selecting just one of the examples with the name **Blink**. It turns the LED on and off with some time delay. You can select any other example from the list.

Step 6: Select your Arduino board.

To avoid any error while uploading your program to the board, you must select the correct Arduino board name, which matches with the board connected to your computer.

Go to Tools -> Board and select your board

CHAPTER 7

PROJECT IMPLEMENTATION

7.1 Implementation:

A DC motor is a machine that converts electrical energy into mechanical energy. The operation is based on simple electromagnetism, i.e. “when a current carrying conductor is placed in a external magnetic field it experiences a force which is proportional to the current and the external magnetic field.” A torque is generated by the magnetic reaction and the armature revolves and this induces a voltage in the armature windings which is opposite in direction to the outside voltage applied to the armature, when current is passed through the armature of the DC motor, and hence is called back voltage or counter e.m.f. The back voltage rises till it becomes equal to the applied voltage as the motor rotates faster, the speed at which the DC motor rotates depends on two factors - the armature current as well as the strength of the magnetic field acting on the armature.

The code for the system is written in Arduino IDE. The code reads the temperature from the LM35 sensor and displays it on the I2C display. It also reads the state of the two push buttons and adjusts the PWM signal to control the motor speed accordingly. If the temperature goes above 45 degrees Celsius, the code activates the relay to turn on the fan. The DC motor speed control using PWM (Pulse Width Modulation) through Arduino Uno and L293D driver IC involves the following steps:

- Connect the Arduino Uno to the computer using a USB cable and open the Arduino IDE.
- Design the circuit y connecting the necessary components, including Arduino Uno, L293D driver IC, DC motor, LM35 temperature sensor, and a 5V SPDT relay.
- Upload the required program to the Arduino Uno to control the speed of the DC motor based on the PWM signal.
- Set the desired speed using the program and observe the response of the DC motor

7.2 ADVANTAGES AND APPLICATION

7.21 Advantages:

i. Precise Motor Control: The DC motor speed controller enables precise control over the rotational speed of the motor, allowing for accurate adjustments to meet specific

application requirements.

ii. Versatility: The bidirectional control feature of the DC motor speed controller provides versatility, enabling seamless control over motor rotation in both directions.

iii. Real-time Temperature Monitoring: The temperature sensor integrated with Arduino facilitates real-time temperature monitoring, ensuring timely intervention in critical scenarios.

iv. Customization and Expandability: The Arduino platform allows for easy customization and integration of additional sensors, modules, or displays, expanding the functionality of the system.

v. User-Friendly Interface: The system provides a user-friendly interface for easy configuration of motor speed settings, temperature thresholds, and notification preferences.

vi. Eco-Speed Control: Some modern systems optimize vehicle speed not only for safety but also for fuel efficiency. These systems adjust the vehicle's speed to minimize fuel consumption based on real-time driving conditions and road gradients.

vii. Cost-Effective Solution: The Arduino-based system offers a cost-effective solution compared to proprietary motor controllers and temperature monitoring systems.

viii. Smart Traffic Signals and Vehicle Coordination: Research has explored how smart traffic signals, coupled with vehicle-to-infrastructure (V2I) communication, can help vehicles adjust their speeds proactively in response to traffic flow, congestion, and accidents.

7.22 Applications:

Safety Applications

1. Automatic Emergency Braking (AEB)
2. Lane Departure Warning (LDW)
3. Forward Collision Warning (FCW)
4. Adaptive Cruise Control (ACC)
5. Electronic Stability Control (ESC)

Efficiency and Fuel Economy

1. Cruise Control
2. Speed Limiters

3. Eco-Drive Systems
4. Fuel-Efficient Driving Assist
5. Real-Time Traffic Optimization

Autonomous and Semi-Autonomous Vehicles

1. Autonomous Cruise Control
2. Traffic Jam Assist
3. Highway Pilot
4. Lane Centering
5. Autonomous Emergency Steering

SUMMARY

The Vehicle Speed Control System project focuses on developing a system that can automatically manage and regulate the speed of a vehicle to enhance safety, improve efficiency, and ensure compliance with speed limits. The system integrates various technologies, including motors, sensors, and advanced control algorithms, to provide real-time monitoring and adjustments of the vehicle's speed based on its surroundings, traffic conditions, and road regulations.

Challenges and Future Work:

Sensor Accuracy: The system's performance depends on the accuracy of the sensors. Issues like sensor malfunction, poor weather conditions (e.g., rain, fog), or environmental interference could affect the system's reliability.

Driver Acceptance: Fully autonomous speed control may face resistance from drivers who prefer manual control over vehicle speed. Therefore, a balance between manual override and automation is crucial for widespread adoption.

Integration with Traffic Systems: The future of vehicle speed control lies in integrating with smart traffic systems and vehicle-to-vehicle (V2V) or vehicle-to-infrastructure (V2I) communication, enabling more dynamic speed regulation based on real-time traffic conditions.

Safety: By using real-time data from sensors and adaptive control algorithms, the system can prevent accidents, maintain safe distances from other vehicles, and adapt to changes in the driving environment, such as traffic or obstacles.

Regulatory Compliance: system ensures that the vehicle does not exceed speed limits and adjusts speed accordingly, which is particularly useful in environments with variable speed limits (e.g., urban streets, school zones).The

Autonomous Operation: In future developments, the vehicle speed control system could be fully integrated into autonomous driving systems, where it manages not only speed but also other critical functions such as steering and braking.

CONCLUSION

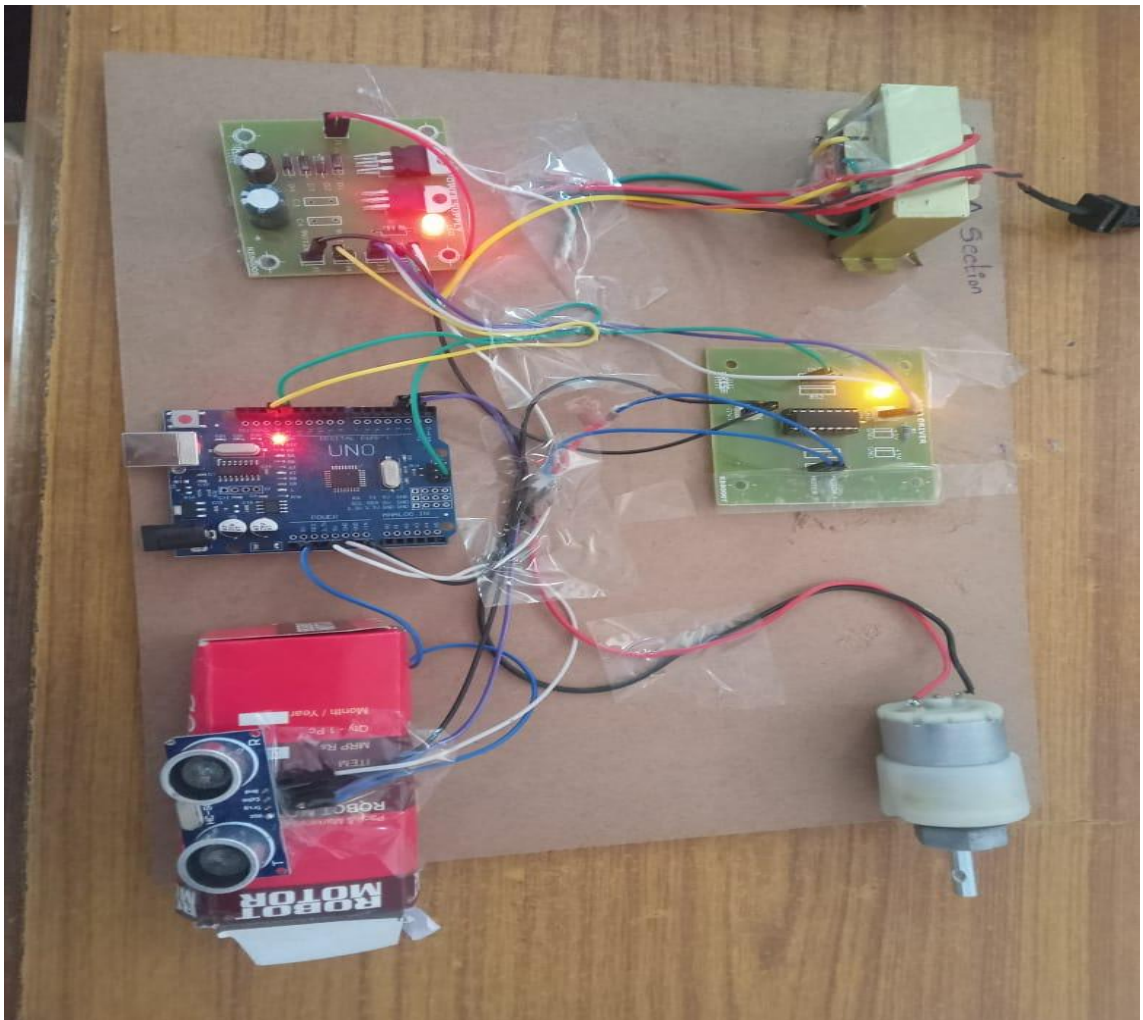
The project “Automatic Speed Control Of Vehicle Using Ultrasonic Sensors” has been successfully designed and tested. Integrating features of all the hardware components used have developed it. Presence of every module has been reasoned out and placed carefully thus contributing to the best working of the unit. Secondly, using highly advanced IC’s and with the help of growing technology the project has been successfully implemented.

The **Vehicle Speed Control Project** presents a significant advancement in enhancing road safety, improving fuel efficiency, and optimizing traffic flow through intelligent technology. By integrating modern sensors, electronic control systems, and real-time data processing, this project addresses several key issues associated with speeding, such as accidents, fuel wastage, and environmental impact.

The core objective of the project was to design a system that could automatically regulate the speed of a vehicle, ensuring compliance with traffic regulations while maintaining driving comfort. Through features like automatic speed limiting, adaptive cruise control, road sign recognition, and GPS integration, the system can adjust the vehicle’s speed based on road conditions, traffic patterns, and regulatory requirements.

RESULT

Vehicle speed control technologies have evolved from simple cruise control to sophisticated, integrated systems that improve safety, efficiency, and performance. As autonomous driving becomes more common, the future of vehicle speed control will continue to be shaped by advancements in AI, sensor technology, and vehicle-to-vehicle communication. These technologies are expected to create safer, more efficient roads while minimizing environmental impact.



REFERENCES

- [1] Arduino: A Technical Reference: A Handbook for Technicians, Engineers, and Makers M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.
- [2] Arduino Reference: Syntax, Concepts, and Examples - 1nd Edition Claudia Alves.
- [3] Arduino: 2020 Step-by-Step Guide for Absolute Beginners. Everything You Need to Know about Arduino Nick.
- [4] Dr. Lavanya Dhanesh, Dr.S.Deepa, (2020), "Enhanced and Energy-Efficient Program Scheduling for Heterogeneous Multi-Core Processors System", Lecture Notes in Electrical Engineering ,2020, 665, pp. 737-747 (Scopus)
- [5] Dr.S.Deepa, R.Madubala, J.Hindhupriya,(2020), "Relay Coordination Using Etap", International Journal Of Advance Research And Innovative Ideas In Education, Vol.6, No.4, Pp. 1180-1188. 2020
- [6] Dr.S.Deepa , Dr.Lavanya Dhanesh (2019) , "Optimal Fuzzy Controller For Power Quality Improvement Of Dynamic Voltage Restorer Using Bacterial Foraging Algorithm", " International Journal of Advanced Science and Technology" Vol. 28, No. 19, (2019), pp. 10-15
- [7] Dr.Lavanya Dhanesh (2019) "Solar Panel Monitoring System Using Smart Phone Technology" in the International Journal of Advanced Research in Electrical, Electronics Electronics and Instrumentation Engineering, ISSN 2278-8875, Vol. 8, Issue 3, March 2019
- [8] P.S RAMAPRABHA, A. Mohamed Inam ul-Hasan, M, Manuneethy Cholan, D. Senthil Kumar, VG. Samraj, "SMART DRINKING WATER DISPENSER USING DUAL AXIS SOLAR TRACKER" Vol-6 Issue-5 2020 IJAR II ISSN(O)-2395-4396 pp 226-241
- [9] Deepa S. Rajapandian.S., (2011), "Simulation of Dynamic Voltage Restorer Using Embedded Z source inverter", CIIT International journal of Automation and Autonomous

System, Vol.3, No.4, pp.178-182.

[10] Lavanya Dhanesh, Dr.P.Murugesan (2018), “A Novel Approach in Scheduling Of the Real- Time Tasks In Heterogeneous Multicore Processor with Fuzzy Logic” in the International Journal named “International Journal of Power Electronics and Drive System (IJPEDS)” ISSN: 2088-8694, Vol. 9, No. 1, pp. 80-88.

[11] P.S.RAMAPRABA “SENSORLESS BLDC MOTOR DRIVE WITH COMMUTATION CORRECTION” IJARIE Vol-5 Issue-2 2019-ISSN 2395-4396 DOI

[12] Lavanya Dhanesh, Dr.P.Murugesan (2015), “Power Saving of The CPU By Improving The Performance of The Real - Time System Kernal Using The PSCPPTS Algorithm” in the International Journal named “International Journal of Applied Engineering Research”ISSN 0973-4562 Volume 10, Number 5 (2015) pp. 12465-12473.

[13] Lavanya Dhanesh, Dr.P.Murugesan (2015) , “Analysing the WCET by implementing Cyclic Priority Pre- Emptive Task Scheduling algorithm” in the International Journal named “Far East Journal Of Mathematical Sciences” ISSN 0972-0871 Volume 97,Number 6, 2015 pp. 667-688 .

APPENDIX

Code

```
#include <LiquidCrystal.h>

LiquidCrystal lcd(A0,A1,A2,A3,A4,A5);
const int trigP = 2; //D4 Or GPIO-2 of nodemcu
const int echoP = 3; //D3 Or GPIO-0 of nodemcu

long duration;
int distance;

void setup() {
  pinMode(trigP, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoP, INPUT); // Sets the echoPin as an Input
  Serial.begin(115200);
  pinMode(13,OUTPUT);
  pinMode(12,OUTPUT);
  pinMode(11,OUTPUT); // Open serial channel at 9600 baud rate
  pinMode(10,OUTPUT);
  lcd.begin(16,2);
  lcd.setCursor(0, 0);
  // Print a message to the LCD.
  lcd.print("Vehicle speed sys");
  lcd.setCursor(0, 1);
  // Print a message to the LCD.
  lcd.print("          ");
}

void loop() {
  digitalWrite(trigP, LOW); // Makes trigPin low
```

```

delayMicroseconds(2);    // 2 micro second delay

digitalWrite(trigP, HIGH); // trigPin high
delayMicroseconds(10);    // trigPin high for 10 micro seconds
digitalWrite(trigP, LOW);  // trigPin low

duration = pulseIn(echoP, HIGH); //Read echo pin, time in microseconds
distance= duration*0.034/2;      //Calculating actual/real distance

Serial.print("Distance = ");    //Output distance on arduino serial monitor
Serial.println(distance);
delay(1000);
if(distance<35)
{
analogWrite(13,0) ;//Pause for 3 seconds and start measuring distance again
analogWrite(12,0) ;
analogWrite(11,0) ;//Pause for 3 seconds and start measuring distance again
analogWrite(10,0) ;
lcd.setCursor(0, 0);
// Print a message to the LCD.
lcd.print("Vehicle speed sys");
lcd.setCursor(0, 1);
// Print a message to the LCD.
lcd.print("stop.....");
}
if(distance>35)
{
digitalWrite(13,1) ;//Pause for 3 seconds and start measuring distance again
digitalWrite(12,0) ;
digitalWrite(11,1) ;//Pause for 3 seconds and start measuring distance again
digitalWrite(10,0) ;
lcd.setCursor(0, 0);
// Print a message to the LCD.
lcd.print("Vehicle speed sys");
lcd.setCursor(0, 1);

```

```
// Print a message to the LCD.  
lcd.print("started.....");  
}  
}
```